Final Report for E2E Natural Language Generation Challenge

Ke Liang [#], Sanaz Mahmoodi Takaghaj [#], Sharad Maheshwari[*]

Department of Computer Science and Engineering, The Pennsylvania State University
* Department of Chemical Engineering, The Pennsylvania State University

Introduction

Natural Language Generation (NLG) is a process of transforming structured data into natural language. This process finds a wide-ranging application ranging from automatic report generation to producing content for a web or mobile application. It further is an essential component piece towards development of conversational agents. NLG also has theoretical applications in the field of developing translation systems, psycholinguistic modelling etc.

The E2E approach for natural language generation aims to rapidly develop the components of natural language generation in different domains by reducing struggles encountered for annotation. This is achieved by simultaneously learning sentence planning and surface realization from non-aligned data. Further, E2E NLG systems overcome the need for semantic alignment between MRs and the corresponding natural language reference texts and instead are trained on a parallel dataset.

For the shared E2E NLG Challenge task [1], crowd-sourced data of ~50k instances in the restaurant recommendation domain is used to generate the utterance from a given MR. The novelty of this shared task lies in the extensive size of the dataset. It is significantly larger than the datasets used in previous E2E NLG approaches that have been limited to small, de-lexicalized data sets, e.g. BAGEL[2], SF Hotels/ Restaurants [3], or RoboCup [4]. These types of datasets limit the end-to-end approaches to generate the rich dialogue and discourse phenomena. The larger dataset such as the one used for the E2E NLG shared task aims to overcome these limitations. However, such a large dataset poses significant new challenges such as open vocabulary, complex syntactic structures and diverse discourse phenomena.

In the interim report we presented the implementation of the sequence-to-sequence (seq2seq) framework-based baseline TGen model [5] for the shared E2E NLG task. The performance of TGen model and other seq2seq models was significantly better than other rule based, and template-based models submitted as a part of the Challenge [1]. Overall best performance was also achieved by a seq2seq based SLUG system[6]. From our work during the interim project, we identified two strategies - ***data augmentation and modifying model framework***, that can help to boost the performance for the shared E2E NLG task.

To augment the original E2E NLG dataset, we used three approaches. First, we extended our training dataset by permuting the slot ordering in the MR's [7] while keeping the same reference utterance for all the permutations. In the second approach, some conjunctions were added to augment the relationship inside of the sentence, such as the contrast or concession relationships which include ''but",

“however” and “yet”. Lastly, the dataset was preprocessed to perform utterance splitting and augment the dataset with new pseudo-samples whose utterances are one sentence long. The MR of each pseudo-sample contains only slots mentioned in the corresponding sentence.

To modify the model architecture, we used tensor-to-tensor (t2t) framework instead of seq2seq framework. This shift in model framework resulted in switching from LSTM-based model to Transformer-type model [8].

In this report, we present our work aimed to implement the two strategies highlighted above. In Section 2, we explain the data preparation including delexicalization and the data augmentation approaches implemented in this work. Section 3 details about model architectures including the baseline TGen and the Transformer models. We explain the Transformer model architecture and also highlight the improvements of Transformer models over LSTM based models. The training parameters and data splitting are explained in the Section 4 and 5 respectively. In Section 6 we present the details of different automatic metrics used to evaluate the model performance. The performance of both approaches, i.e. data augmentation and changing model architecture are then compared to the originally submitted models for the E2E NLG Challenge task in Section 7. Though neither of the strategies were able to better the performance for the task as compared to the SLUG model [6], the overall winner system of the E2E NLG Challenge, we see significant reduction in training time with Transformer models. Based on the results, we conclude that the seq2seq model is better at this NLG task if training time is not a concern.

Data Preparation

Original E2E NLG Challenge Dataset

The original dataset for the E2E NLG task comprises ~50k instances in the restaurant recommendation domain. These instances comprise an MR which is used to generate the utterance in human language form. On an average each MR corresponds to ~8.6 reference utterance and thus resulting in approximately 6k unique MRs. The original dataset comprises of 8 types of slot values as shown in Table 1[1]:

Attribute Data	Data Type	Example value
name	Verbatim string	The Eagle, ...
eatType	dictionary	restaurant, pub, ...
familyFriendly	Boolean	Yes / No
priceRange	dictionary	cheap, expensive, ...
food	dictionary	French, Italian, ...
near	verbatim string	market square, Cafe Adriatic, ...

area	dictionary	riverside, city center, ...
customerRating	enumerable	1 of 5 (low), 4 of 5 (high), ...

Table 1 List of attributes in E2E NLG dataset along with data type and example of each

Delexicalization

One of the key challenges posed with a larger dataset such as the one used in this shared task is open vocabulary. As the domain covers different restaurants, there is a possibility of training data never seeing some of the vocabulary (like the name of a restaurant or cuisine). This in turn can limit the utility of the model. One way to handle this challenge of open vocabulary in NLG systems is to use delexicalization. **In all the models used in this work, partial delexicalization is used.** For the TGen model delexicalization of the slot values for "name" and "near" slots is performed. This delexicalization is achieved using a pre-processing script which simply replaces the slot values corresponding to the slots "name" and "near" in original MRs by "X-name" and "X-near" respectively. This allows the model to bypass the uncommon words corresponding to the proper-nouns present in the original MR-utterance pair. Once the model is trained, a post-processing script can be used to replace these slot values in the utterance by corresponding values of the "name" and "near" in the original MR. A simple example to illustrate this delexicalization process is shown Table 2:

MR:	name[Green Man], food[French], priceRange[more than £30], area[city centre], familyFriendly[no], near[All Bar One]
Lexicalized Utterance:	Green Man is a French restaurant in the city centre. It is not child friendly and is located near All Bar One. It costs more than thirty pounds.
Delexicalized:	X-name is a french restaurant in the city centre . it is not child friendly and is located near X-near . it costs more than thirty pounds .

Table 2 An example to explain the delexicalization process of slots "name" and "near"

For the t2t model, which is based on the approach taken by the SLUG system [6], the "food" slot was also delexicalized. In Figure 1, a code snippet to implement delexicalization is shown.

```
def delexicalize_tokenize_mr(mr):
    features = ["eatType", "food", "priceRange", "customer rating", "area", "familyFriendly", "near"]

    delex_mr = []
    for feature in features:
        delex_mr.append(feature)
        value = extract_feature(mr, feature)
        if value != "":
            if feature == "food":
                value = "FOOD_TAG"
            if feature == "near":
                value = "NEAR_TAG"
            if feature == "name":
                value = "NAME_TAG"
        delex_mr.append(value)

    return delex_mr
```

Figure 1 Code snippet to perform delexicalization

Data Augmentation 1 - Slot permutations

Slot values can alter the skeleton of the utterance as suggested by Nayak et al [7]. Thus, as a first approach to augment the original dataset, the slots in the training MRs were permuted. Among all permutations, 5 random permutations were selected with the same reference utterance as new pseudo samples. This thus increased training set size six-fold. An example to show slot permutation is shown in Table 3. In the original dataset, the slot order was fixed both in the training and test set. It is anticipated that slot permutation on the training data would help make the model more robust. However, this can also misdirect the model as the training would happen on the slot orders that are majorly not seen in the test set.

Original MR	name[Alimentum],food[Chinese], priceRange[cheap], area[riverside], familyFriendly[yes]
Permutation 1	priceRange[cheap],area[riverside], familyFriendly[yes],name[Alimentum], food[Chinese]
Permutation 2	priceRange[cheap],familyFriendly[yes], food[Chinese],name[Alimentum], area[riverside]
Permutation 3	area[riverside], food[Chinese], familyFriendly[yes],name[Alimentum], priceRange[cheap]
Permutation 4	food[Chinese], priceRange[cheap], familyFriendly[yes],area[riverside],name[Alimentum]
Permutation 5	familyFriendly[yes], area[riverside], priceRange[cheap], food[Chinese],name[Alimentum]

Table 3 An example to show slot permutations approach to augment training dataset

Data Augmentation 2 - Logic of the sentence

When generating real sentences, we often use certain words such as “yet”, “however”, “but” etc. to make the logic of the sentence clearer. This is the idea behind our second approach to augment the dataset. Based on such words, more MR is added to the original MR so as to better represent the logic of the sentence. This is achieved by adding auxiliary tokens indicating desired discourse phenomena in the corresponding utterances. In this implementation, the concession and contrast relationships are chosen as the new MR by searching whether there is “however”, “yet” or “but” in the reference utterance. This is explained using the example in Table 4:

Ref	Located near the Rainbow Vegetarian Café, it's inexpensive and family friendly, but the Chinese food at Strada has a low customer rating.
MR in Original dataset	name[Strada], food[Chinese], priceRange[less than £20], customerRating[low], familyFriendly[yes], near[Rainbow Vegetarian Café]

MR in the Augmented dataset	name[Strada], food[Chinese], priceRange[less than £20], customerRating[low], familyFriendly[yes], near[Rainbow Vegetarian Café], __contrast__[familyFriendly customerRating]

Table 4 An example to illustrate the data augmentation to incorporate sentence logic.

The first line in each table shows the reference sentence. The MR in the original dataset and the MR in the augmented dataset are shown in the second line and the third line. Based on the logic inside of the sentence, the logic token is added in the MR. The example shown here is for the contrast relationship. The attributes “customerRating” and “familyFriendly” in this example are the key words for the relationship.

Data Augmentation 3 - De-noise

While going through the dataset, it was observed that there are some dialogue acts which appear in the MR but have no directly related word in the reference utterance. These dialogue acts can instead misdirect the model and can be considered as “noise”. In this third approach of data augmentation, such attributes were removed in MR with the expectation that it will help improve the model learning. An example to illustrate this data augmentation approach is shown in Table 5:

Ref	Close to Café Brazil, The Cambridge Blue pub serves delicious Tuscan Beef for the cheap price of £10.50. Delicious Pub food. .
MR in Original dataset	name[The Cambridge Blue], eatType[pub], food[English], priceRange[cheap], near[Café Brazil]
MR in the denoise dataset	name[The Cambridge Blue], eatType[pub], priceRange[cheap], near[Café Brazil]

Table 5 An example to illustrate the data augmentation via de-noising MR

The first line the table shows the reference sentence. The MR in the original dataset and the MR in the augmented dataset are shown in the second line and the third line. As it can be seen that there is no mention of attribute *food[English]* in the reference utterance. Thus, in augmented MR, this attribute is removed which might help the system to learn from the MR more clearly.

Model Details

TGen

The TGen model [5] is the baseline model for the E2E NLG Challenge. The model is based on the seq2seq framework with attention with an encoder-decoder recurrent neural network (RNN) architecture. In addition to the standard seq2seq model with LSTM cells, TGen uses beam search for decoding and an LSTM-based re-ranker over the top k outputs, penalizing those outputs that do not verbalize all attributes from the input MR.

Original Transformer

Most of the seq2seq based models submitted for the E2E NLG Challenge used Long Short-Term Memory (LSTM) based Recurrent Neural Network (RNN) framework [9]. While these models yield high performance metrics such as BLEU [10], they had some constraints as they were computation-heavy and also had difficulties maintaining long-term dependencies due to vanishing gradient effect. To overcome some of these limitations of the RNN framework, we used Transformer models which were introduced by Google in late 2017 [8]. These transformer models are more parallelizable and require significantly less time to train. The novelty of these Transformer models stems from the complete removal of recurrence and convolutions while the encoder and decoder parts are connected through an attention mechanism as shown in Figure 2.

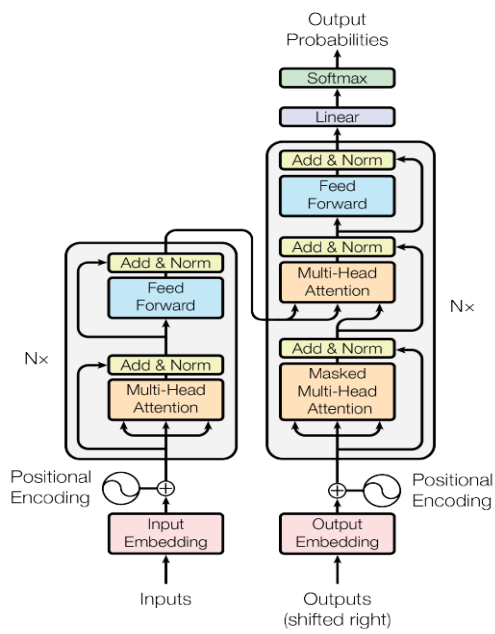


Figure 2 The Transformer model architecture [8]

The encoder part is a stack of N identical layers (shown on the left) where each layer contains two sub-layers: Multi-Head Attention and a fully connected Feed-Forward neural network. The residual connections are employed around each of these sub-layers, followed by a normalization layer (Add & Norm).

The decoder (shown on the right) is also composed of N identical layers. In addition to the two sub-layers in the encoder layer, the decoder inserts a third sub-layer as the Masked Multi-Head Attention, which performs multi-head attention over the output of the encoder stack. The term multi-head attention refers to performing the attention function in parallel on each projected version of queries (Q), keys (K) and values (V) with different, learned linear projections. In this project we used dot-product as the attention function:

$$A(Q,K,V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \qquad (1)$$

where $\frac{1}{\sqrt{d_k}}$ is the scaling factor and is used to prevent the softmax function reaching saturation. The input/output embeddings in Figure 1 represent a token in a d-dimensional space where tokens with similar meaning will be closer to each other. These embeddings however do not encode the relative position of words in a sentence. As a result, Positional Encodings which in fact are sine and cosine functions of different frequencies were added to the input/output embeddings at the bottoms of the encoder and decoder stacks. These positional encodings will compensate for the lack of recurrent and convolutional network in the model and let the model better use the information gained from the relative or absolute position of the tokens in the sequence. These encodings have the same dimension as the input/output embeddings, so that the two can be summed. After adding the positional encoding, words will be closer to each other based on the similarity of their meaning and their position in the sentence, in the d-dimensional space.

While the transformer model is still one of the leading architectures for many Natural Language Processing tasks, we looked at some ways to improve the Transformer model to improve the performance or lower the training time. In particular we looked at the Evolved Transformer which is described in the following section.

Evolved Transformer

A recent research [11] used an evolution-based algorithm to mutate the Transformer architecture to discover an even better one, The Evolved Transformer (ET) model. The Neural Architecture Search (NAS) algorithm looked for the best architecture in the given search space based on the constraints of the model such as the number of layers, maximum number of parameters, etc.

Defining the search space was a challenge as if the space was too broad, the algorithm might not have converged and might not have found a better model in a reasonable amount of time. On the other hand, a space that was too narrow would reduce the probability of finding an innovative model that would outperform the hand-crafted models.

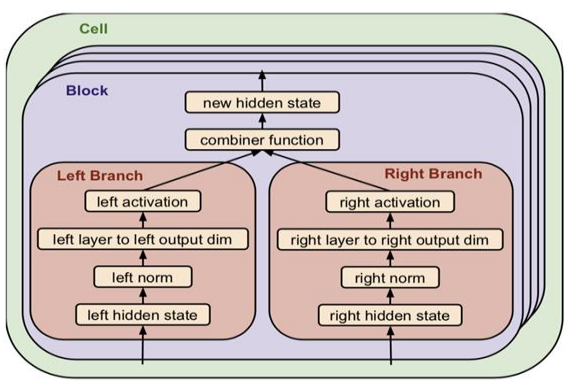


Figure 3 Evolved Transformer Stackable Cell Architecture [11]

The algorithm searched for two types of cells - one for the encoder with six copies (blocks) and another for the decoder with eight copies and the optimal model is built by stacking the same cell architecture several times. Each block includes two branches of operations as shown in Figure 3.

The new Encoder-Decoder blocks for the evolved transformer models are compared with the Encoder-Decoder blocks of the original transformer model in the Figure 4 [11].

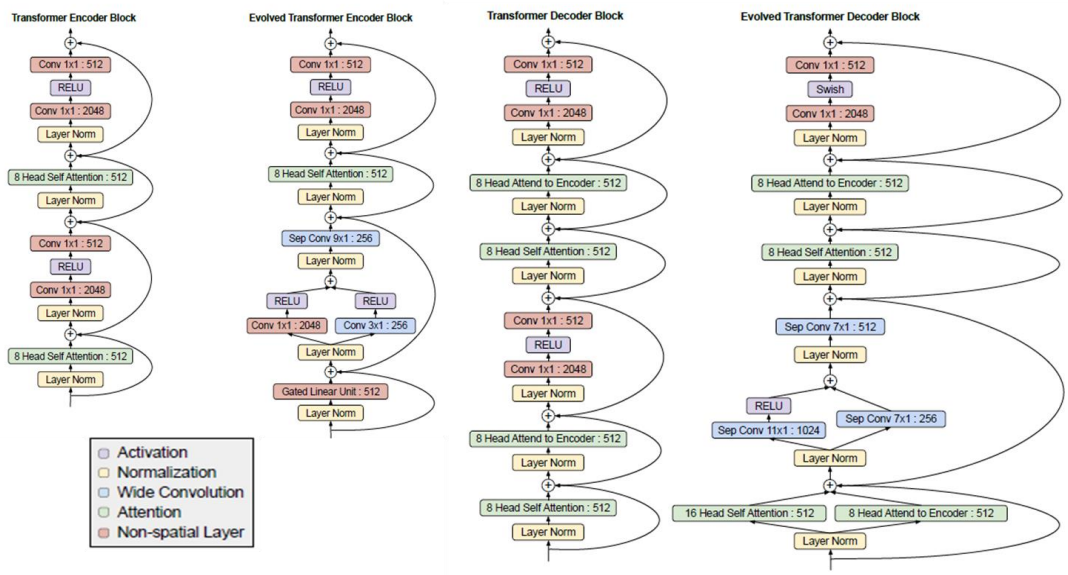


Figure 4 Encoder/ Decoder Blocks- Transformer and Evolved Transformer architecture cells [11]

Training parameters

Seq2seq based TGen model

Developed by Dusek and Jurcicek [5], the TGen model uses seq2seq technique with attention within the framework of encoder-decoder recurrent neural network (RNN) architecture. TGen model also uses beam search during the decoding process along with an n-best list re-ranker to suppress irrelevant information in the outputs. The main seq2seq model is trained by minimizing cross entropy using the Adam algorithm [12] in direct token-by-token generation of surface strings; the re-ranker is trained to detect the presence of all attributes from the input MR. We use a learning rate of 0.0005, number of hidden units 128, cell size 50, batch size 20, beam size 10, maximum encoder and decoder lengths 10 and 80, respectively, and up to 20 passes through training data with early stopping when we are running model with the augmentation method. The re-ranker uses the same parameters, except for a higher learning rate (0.001). Different training parameters were also tested such as changing the learning rate to 0.0005 with 0.03 decay, number of hidden units to 256 which didn't alter the model performance.

T2T based model (Transformer)

The Transformer-based model was trained with two hidden layers comprising 256 hidden units in each layer. Multi-head attention with dot-product attention function is used with 8 heads and the dropout rate of 0.2. The model is trained for a maximum of 10,000 steps with the Adam algorithm [12] to minimize the cross entropy. In order to ensure that the learning process doesn't diverge and is faster and stable, we used a learning rate scheduler with the base learning rate of 0.002 and 1000 warmup steps where the learning rate grows linearly and then follows cosine decay for the remainder of the steps.

Train/Test split

The original E2E NLG data was split into training, validation and test sets in an 82-9-9 ratio, and the same ratio is also used for models in this work (except when slot permutation approach is used). It is ensured that the MRs in the test set are all previously unseen, i.e. none of them overlaps with training/development sets, even when restaurant names are removed unlike the SFRest data [3]. During the original development of the TGen model, the authors used 10-fold cross-validation. The authors also reserve 10 DAs exclusively for validation and selected the hyperparameters based on the highest BLEU score obtained for this validation set [5]. This again helps to build a model that can interpolate well for the unseen data. The same approach is used in this work.

Upon slot permutation, the training set became six-times that of the initial training set. For training the model with data set augmented with slot permutation, the training, validation and testing splits were in the ratio 96:2:2. The idea with slot permutation is to make the model more robust and enhance its ability to generalize (More and diverse the data, better the learning).

Evaluation Metrics

BLEU

BLEU or Bilingual Evaluation Understudy [10] is a score used to evaluate the word-overlap between system output and references. Originally, the BLEU score was devised to evaluate the quality of machine-translated text from one language to another. It is the harmonic mean of n-gram precisions of the system output with respect to human-authored reference sentences, with $n \in [1, ..., 4\}]$ lowered by a brevity penalty if the output is shorter than references. The n-gram precisions are proportions of n-grams in the system output that can be matched in any of the reference sentences. Repeated n-gram matches are clipped to the maximum number of times the n-gram occurs in any single reference. In the interim report, some deficiencies of the BLEU metrics were highlighted. Following metrics attempt to solve those deficiencies.

NIST

NIST [13] is a word-overlap metric developed by the US National Institute of Standards and Technology to overcome the limitations of the BLEU score. It is based on the BLEU score with some alteration where it weighs the penalties for the mis-matched n-grams based on the probability of the occurrence of the n-gram - more common n-grams are penalized lower while the mis-match on rare n-grams is more heavily penalized. This approach of penalizing rarer words more heavily allows for capturing function words which are often rare but are crucial to convey the meaning of the generated text. Higher penalties on rarer n-grams however, restricts the use of synonyms in the generated text.

METEOR

METEOR or Metric for Evaluation of Translation with Explicit Ordering (METEOR) [14] was developed to overcome some of the limitations of the BLEU score and also to produce better correlation with human judgement at the sentence level, unlike BLEU which aims to optimize the correlation at the corpus level. METEOR score is based on the harmonic mean of unigram precision and recall. Recall is weighted higher than precision in the evaluation. It is unique as it includes stemming (process of reducing the word to their base or root form - generally a written form) and synonym matching along with exact word matching.

ROUGE_L

ROUGE or Recall-Oriented Understudy for Gisting Evaluation [15] is a modification of the BLEU score that focuses on recall rather than precision. In other words, it looks at how many n-grams in the reference translation show up in the output, rather than the reverse. There are many variants of ROUGE metric[16]. The ROUGE variant that is used for E2E Challenge is ROUGE_L which stands for *Longest Common Subsequence (LCS) [15]* based statistics. It takes into account sentence level structure similarity naturally and identifies longest co-occurring in sequence n-grams automatically

There are several limitations that have been pointed out by several studies for using ROUGE metrics. In the context of the summarization task, Louis and Nenkova [17] observed that ROUGE correlation with human scores drops significantly with fewer reference summaries. They also found that score drops significantly when looking at an individual summarization task versus system comparison across many summarization tasks. It has also been found that ROUGE metric tends to prefer long sentences.

CIDE_r

CIDE_r or Consensus-based Image Description Evaluation is an automatic evaluation metric [18], initially developed as a protocol to evaluate automatic generated image caption based on consensus. It

rewards a sentence for being similar to the majority of human written descriptions. It evaluates the importance of n-grams by computing Term Frequency Inverse Document Frequency (TF-IDF) and gives more weightage to important n-grams. It has higher correlation with human consensus scores compared to other metrics.

Performance Results (for all the metrics)

All data augmentation approaches were tested with the TGen model. Transformer models were trained using the original E2E NLG dataset. Due to the time limitation, Transformer model only with Sentence Logic data augmentation approach could be tested.

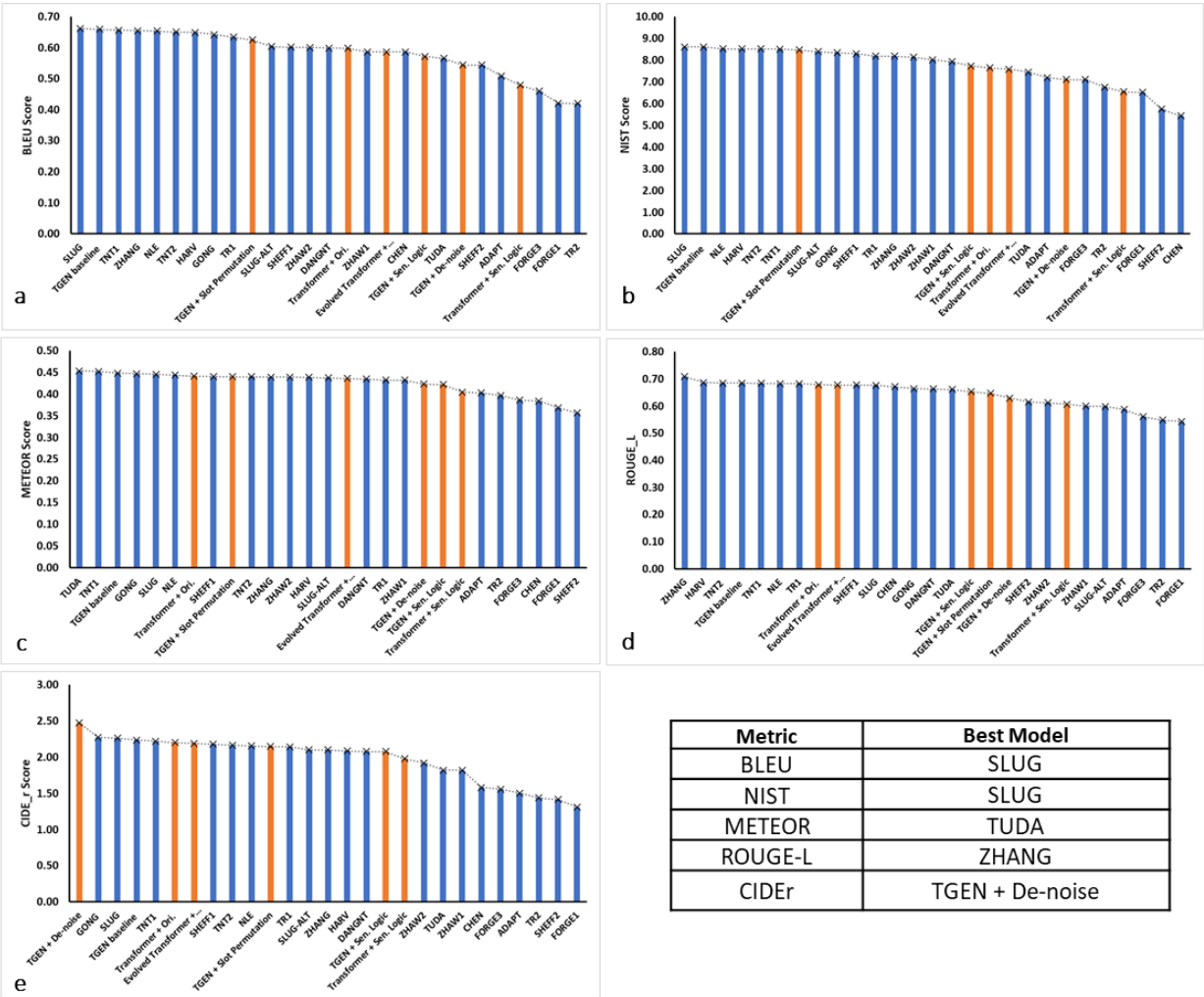
In Table 6, the results of the original 22 systems and the models implemented in this work (6 systems) are compared based on the automatic evaluation metrics. The results in Table 6 are sorted in the decreasing order of the normalized average score of five automatic evaluation metrics obtained by the models. The results of the models implemented in this work are highlighted in orange.

Rank	System	BLEU	NIST	METEOR	ROUGE-L	CIDEr	Norm. Avg
1	TGEN baseline (Novikova et al., 2017b)	0.6593	8.6094	0.4483	0.685	2.2338	0.5754
2	SLUG (Juraska et al., 2018)	**0.6619**	**8.613**	0.4454	0.6772	2.2615	0.5744
3	TNT1 (Oraby et al., 2018)	0.6561	8.5105	0.4517	0.6839	2.2183	0.5729
4	NLE (Agarwal et al., 2018)	0.6534	8.5300	0.4435	0.6829	2.1539	0.5696
5	TNT2 (Tandon et al., 2018)	0.6502	8.5211	0.4396	0.6853	2.1670	0.5688
6	HARV (Gehrmann et al., 2018)	0.6496	8.5268	0.4386	0.6872	2.0850	0.5673
7	ZHANG (Zhang et al., 2018)	0.6545	8.1840	0.4392	**0.7083**	2.1012	0.5661
8	GONG (Gong, 2018)	0.6422	8.3453	0.4469	0.6645	2.2721	0.5631
9	TR1 (Schilder et al., 2018)	0.6336	8.1848	0.4322	0.6828	2.1425	0.5563
10	**TGEN + Slot Permutation**	0.6248	8.4773	0.4400	0.6473	2.1477	0.5549
11	SHEFF1 (Chen et al., 2018)	0.6015	8.3075	0.4405	0.6778	2.1775	0.5537
12	**Transformer**	0.5975	7.6537	0.4408	0.6794	2.1996	0.5406
13	DANGNT (Nguyen and Tran, 2018)	0.5990	7.9277	0.4346	0.6634	2.0783	0.5395
14	SLUG-ALT (late, Juraska et al., 2018)	0.6035	8.3954	0.4369	0.5991	2.1019	0.5378
15	**Evolved Transformer**	0.5861	7.585	0.4361	0.6781	2.1906	0.5356
16	ZHAW2 (Deriu and Cieliebak, 2018)	0.6004	8.1394	0.4388	0.6119	1.9188	0.5314
17	**TGEN + Sentence Logic**	0.5723	7.7345	0.4218	0.6532	2.0772	0.5257
18	TUDA (Puzikov and Gurevych, 2018)	0.5657	7.4544	**0.4529**	0.6614	1.8206	0.5215
19	ZHAW1 (Deriu and Cieliebak, 2018)	0.5864	8.0212	0.4322	0.5998	1.8173	0.5205
20	**TGEN + De-noise**	0.5438	7.1104	0.4231	0.6302	**2.4687**	0.5110
21	ADAPT (Elder et al., 2018)	0.5092	7.1954	0.4025	0.5872	1.5039	0.4738
22	**Transformer + Sentence Logic**	0.4797	6.5482	0.4044	0.6068	1.9789	0.4687
23	CHEN (Chen, 2018)	0.5859	5.4383	0.3836	0.6714	1.5790	0.4685

24	FORGE3 (Mille and Dasiopoulou, 2018)	0.4599	7.1092	0.3858	0.5611	1.5586	0.4547
25	SHEFF2 (Chen et al., 2018)	0.5436	5.7462	0.3561	0.6152	1.4130	0.4462
26	TR2 (Schilder et al., 2018)	0.4202	6.7686	0.3968	0.5481	1.4389	0.4372
27	FORGE1 (Mille and Dasiopoulou, 2018)	0.4207	6.5139	0.3685	0.5437	1.3106	0.4231

Table 6 Performance comparison of different models in this work with other E2E NLG participants

Among the models implemented in this work, TGen model with slot permutation of the train-set MRs ranks best at #10. The normalized average score for this model is 0.5549 as compared to the maximum of 0.5754 obtained by the TGen model with original dataset. The Transformer model with original dataset is ranked #12. The performance of all the models is also analyzed based on individual automatic metric as shown in Figure 5 so as to better understand the characteristics of model output.



Metric	Best Model
BLEU	SLUG
NIST	SLUG
METEOR	TUDA
ROUGE-L	ZHANG
CIDEr	TGEN + De-noise

*Figure 5 Comparison of model performance based on decreasing order of a) **BLEU** b) **NIST** c) **METEOR** d) **ROUGE_L** e) **CIDEr.** Orange bar indicates the models implemented in this work. The table shows best model corresponding to each automatic metric value alone.*

TGen + Slot permutation ranks #10 and #7 based on the BLEU and NIST scores respectively. This indicates that word overlapping is captured well even after slot permutations. Since penalizing schemes are different in NIST and BLEU metric, with NIST penalizing common n-grams less, it suggests that this approach is able to capture rare n-grams well. TGen+ De-noise system outperforms all other models based on the CIDEr metric. CIDEr correlates well with human description and improved performance on this metric suggests that removing noise from the MRs help model learn better. The Transformer model ranks #7 based on the METEOR metrics.

Discussion

From the results above, TGen + De-noise model outperforms all the models based on CIDEr metric. This data augmentation approach was ideated with the expectation of removing the noise in the MR so as to have the cleaner MRs which would allow the model to learn to generate cleaner sentences. As CIDer metric correlates better with human-generated description, the de-noising has indeed helped produce cleaner and more human like text generation from the MRs.

The performance achieved by TGen model after data augmentation through slot permutation was almost at par with the original dataset when compared on the basis of BLEU score. This suggests that slot permutation has no significant effect on the model learning. The slight lowering of the BLEU score could be because slot permutation might misdirect the model as now most of the slot ordering is not seen in the test-set. Also, as we significantly increase the size of the training data, slot permutation approach significantly increases the training time. With no significant improvement in the performance, the increased training time is unwarranted.

The overall performance of the TGen + Sentence Logic system is very poor. Though the idea to augment the data this way seems reasonable as adding one more attribute in the MR indicating the sentence logic should improve the syntactic ability of the model. However, this attribute is not present in all the samples. This in turn can make this attribute somewhat a “noise” for the model.

The Transformer did not achieve high BLEU scores as compared to other approaches. Though, it renders one unique strength to the whole process of natural language generation. Seq2seq models are typically computationally intensive and require significantly larger training time. On the contrary, the Transformer based models are highly parallelizable and require fraction of the time required by the seq2seq models. In Table 7, we compare the training time required by the TGen and Transformer models when trained using the original training dataset along with their respective BLEU scores. The TGen model performs only slightly better in terms of BLEU score but training time required by the TGen model is almost 6 times that of the Transformer model. The time required by the Transformer model is less because it is based on the t2t unit instead of the LSTM unit and doesn’t include convolutions or recurrence. The attention mechanism is more powerful and faster than the LSTM. Some improvements are also achieved

on the embedding part as it can realize the real bi-directional embedding which is more powerful than the original LSTM based model. However, on the other hand, the Transformer based models is significantly harder to de-bug.

Model	BLEU Score	Training Time (Minutes)
Tgen Model	0.6593	358
Transformer	0.5975	51
Evolved Transformer	0.5861	118

Table 7 Training time required by different model architecture with the original E2E training dataset

Overall, based on the results and discussions above, the best model from this work, i.e. TGen + Slot permutation, ranks #10 when compared on the basis of overall performance. in the all of the systems. While comparing to the other systems which also use the data augmentation methods such as tnt1 and tnt2, it still not that good. In a word, based on the general performance trend, the baseline model is the best model till now.

Conclusion

In this work, two strategies – **data augmentation and changing model architecture** were implemented with an aim to improve the previously reported performances for the E2E NLG Challenge 2017 [9]. For data augmentation, three approaches were applied independently. In the first approach, MR slots were permuted in the training data as it has been suggested that the slot values and ordering can alter the skeleton of the reference utterance [7]. In the second approach, a new token was introduced in the overall dataset to capture the logic of the spoken sentence often expressed with the words such as “but”, “yet” and “however”. In the third approach, slot attributes that were not used in the reference sentences were removed so as to provide a cleaner MR-utterance pair for the model to learn. These approaches for data augmentation were implemented with the baseline TGen model.

The performance of the TGen + slot permutation did not show any significant improvement. The BLEU score on the test-set dropped very slightly which could be due the misdirection to the model as slot order in testing set now is primarily unseen in the training set. The BLEU score for the other two data augmentation approach implemented with the TGen model dropped more. However, TGen + De-noise outperformed all the models based on the CIDEr score.

Since the performance with data augmentation alone, didn’t change very significantly, model architecture was altered from seq2seq to t2t based architecture, which in turn changed the LSTM based framework to Transformer based framework. Two Transformer based models – Original and Evolved were implemented with the original E2E NLG Challenge dataset. The performance of these Transformer based

models was slightly poor (BLEU score 0.5975 vs. 0.6593). However, these Transformer-based models required as much 6-fold less time as compared to the TGen model.

Future work

1. Using Transformer-XL (eXtra Long) [19] - It improves on the Transformer architecture by being a lot faster and by adding segment-level recurrence as well as relative positional encoding. It can process much bigger sentences and to retain longer-term dependencies than the standard transformer architecture. Learning longer-term dependency occurs via a recurrence mechanism beyond a fixed length (as in the LSTM models) without disrupting temporal coherence.

2. More complex approaches for data augmentation such as the one taken by TNT1 and TNT2 system can also be implemented along with de-noise/slot permutation to better the model performance.

References

[1] O. Dusek, J. Novikova, V. Rieser, Evaluating the state-of-the-art of End-to-End Natural Language Generation: The E2E NLG challenge, Comput. Speech Lang., 59 (2020) 123-156.

[2] F. Mairesse, M. Gasic, F. Jurcicek, S. Keizer, B. Thomson, K. Yu, S. Young, L. Assoc Computat, Phrase-based Statistical Language Generation using Graphical Models and Active Learning, Assoc Computational Linguistics, Somerset, 2010.

[3] T.-H. Wen, M. Ga?i?, N. Mrk?i?, P.-H. Su, D. Vandyke, S. Young, Semantically Conditioned LSTM-based Natural Language Generation for Spoken Dialogue Systems, Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Lisbon, Portugal, 2015, pp. 1711-1721.

[4] D.L. Chen, R.J. Mooney, Learning to sportscast: a test of grounded language acquisition, Proceedings of the 25th international conference on Machine learning, Association for Computing Machinery, Helsinki, Finland, 2008, pp. 128–135.

[5] O. Dusek, F. Jurcicek, Sequence-to-Sequence Generation for Spoken Dialogue via Deep Syntax Trees and Strings, Assoc Computational Linguistics-Acl, Stroudsburg, 2016.

[6] J. Juraska, M. Walker, Characterizing Variation in Crowd-Sourced Data for Training Neural Language Generators to Produce Stylistically Varied Outputs, Proceedings of the 11th International Conference on Natural Language Generation, Association for Computational Linguistics, Tilburg University, The Netherlands, 2018, pp. 441-450.

[7] N. Nayak, D. Hakkani-Tur, M. Walker, L. Heck, To Plan or not to Plan? Discourse Planning in Slot-Value Informed Sequence to Sequence Models for Language Generation, 2017.

[8] N.S. Ashish Vaswani, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin, Attention is all you need, CoRR, (2017).

[9] N. Dušek, Rieser, O. Dušek, J. Novikova, V. Rieser, Findings of the E2E NLG challenge, Proceedings of the Eleventh International Conference on Natural Language Generation, Tilburg, The Netherlands (2018) 322-328.

[10] K. Papineni, S. Roukos, T. Ward, W.-J. Zhu, Bleu: a Method for Automatic Evaluation of Machine Translation, Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Philadelphia, Pennsylvania, USA, 2002, pp. 311-318.

[11] C.L. David R. So, and Quoc V. Le, The evolved transformer, Proceedings of the36th International Conference on Machine Learning (ICML), (2019).

[12] D. Kingma, J. Ba, Adam: A Method for Stochastic Optimization, International Conference on Learning Representations, (2014).

[13] G. Doddington, Automatic evaluation of machine translation quality using N-gram Co-Occurrence statistics, Proc. of Second International Conference on Human Language Technology (HLT), (2002) 138-145.

[14] S.B.A.a. Lavie, METEOR: An automatic metric for MT evaluation with improved correlation with human judgments, Proc. of ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization, (2005) 65-72.

[15] C.-Y. Lin, ROUGE: Recall-oriented understudy for gisting evaluation, Proceedings of 2003 Language Technology Conference (2003).

[16]

[17] A. Louis, A. Nenkova, Automatically assessing machine summary content without a gold standard, Comput. Linguist., 39 (2013) 267–300.

[18] C.L.Z. Ramakrishna Vedantam, Devi Parikh, CIDEr: Consensus-based Image Description Evaluation, arXiv, (2014).

[19] Z. Dai, Yang, Z., Yang, Y., Cohen, W. W., Carbonell, J., Le,Q. V., and Salakhutdinov, R, Transformer-xl: Attentive lan-guage models beyond a fixed-length context, arXiv preprint, (2019).